Week 3:
Data Warehouse Deepdive: Snowflake

Keeyong Han

Table Of Contents

1. Recap of the 2nd Week
2. Key Concepts to Remember
3. Snowflake Deepdive
4. Lab #1 Overview
5. Quiz #1 Format
6. Demo & Homework #2

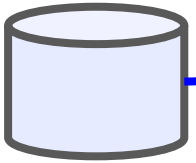
Recap of the 2nd Week

OLTP: OnLine Transaction Processing

OLAP: OnLine Analytical Processing

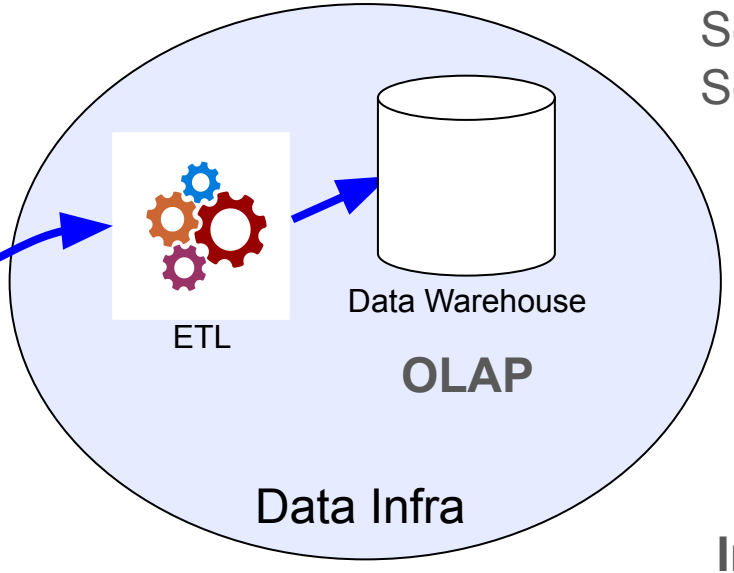
Scale Up

Production DB



OLTP

Scale Up
Scale Out



End Users, Customers

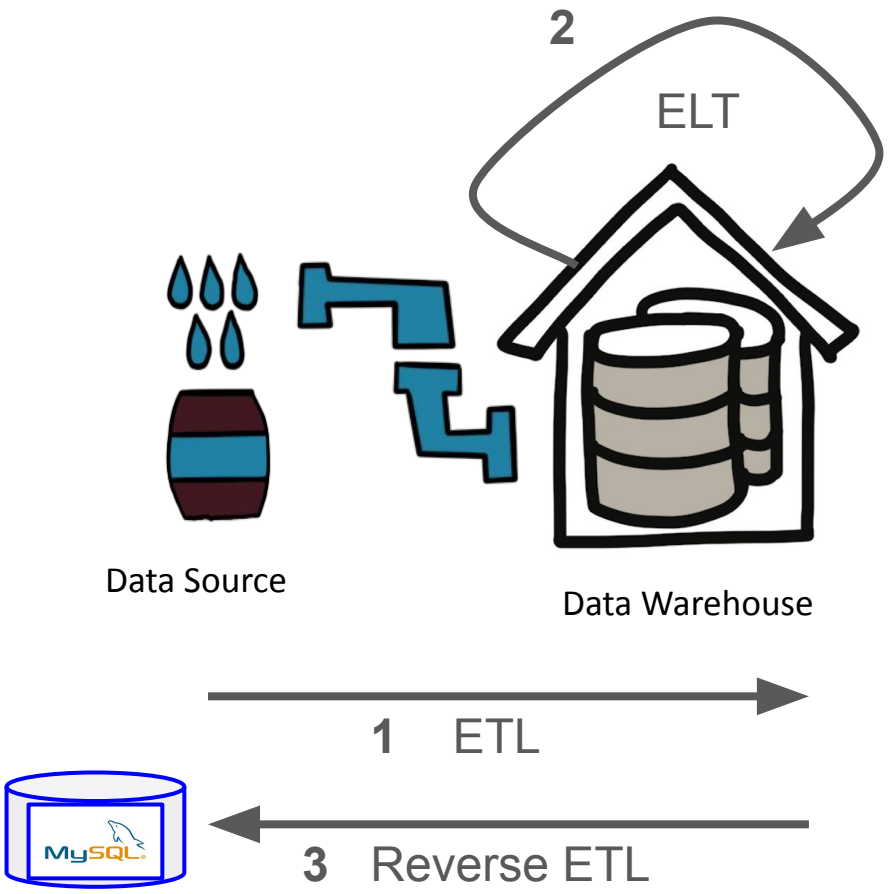
Backend Engineers

Data Engineers

Internal Audience

Types of Data Pipeline

1. ETL
2. ELT (“T”)
3. Reverse ETL

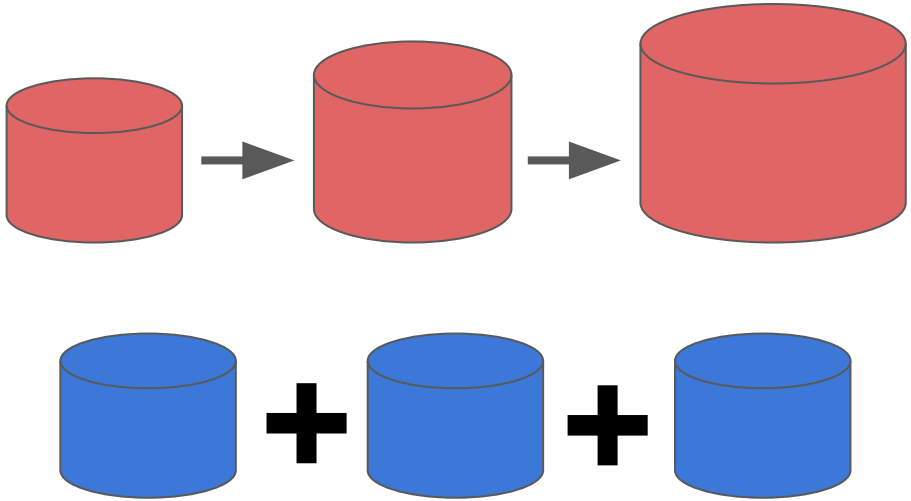


Data Warehouse: Different Options

- **Cloud** vs. On-Premise
- Open Source vs. Proprietary
  - Open Source ones are serviced from Cloud as well
- Fixed Cost vs. Variable Cost
  - CapEx (Capital expenditure) vs. OPEX (Operating expenses)

Scaling Options

- Scale Up: Add more memory, CPUs and disks
- Scale Out: Add more servers



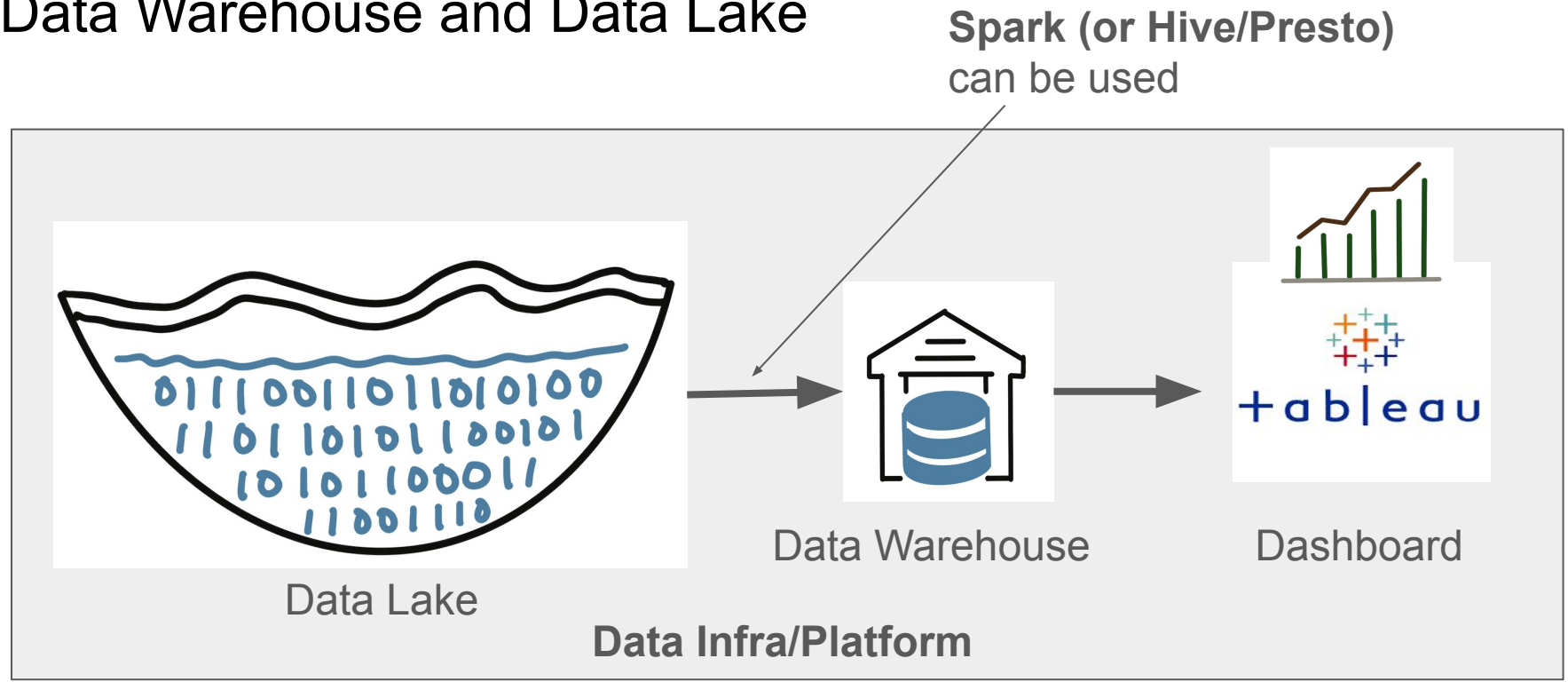
Scale-up

Scale-out

What is Cloud?

- “No Provisioning” & “Pay as You Go”
  - Hardware
    - Servers, Storage, …
  - Software
    - Production databases
    - Cloud Storage
    - Data Warehouses, …
    - Airflow, Spark, …
- AWS, Google Cloud, Azure, …

Data Warehouse and Data Lake



LakeHouse: Data Lake + Data Warehouse (Delta Lake, Iceberg)

Key Concepts to Remember

Columnar Storage vs. Row Storage

- Columnar Storage
  - Stores data by column rather than by record
  - Enables column-level compression and very fast addition or deletion of columns

Table

City	revenue
aa	100
bb	200
cc	300

Row Store

Row1	aa 100
Row2	bb 200
Row3	cc 300

Column Store

City	aa bb cc
revenue	100 200 300

Each is saved as a separate file

Primary Key Uniqueness isn’t guaranteed

- Production databases do guarantee this but not big data based Data Warehouses don’t
- In Data Warehouse, Primary key uniqueness is guaranteed by data people

```
CREATE TABLE adhoc.primary_key_test (
  id int primary key
);
```

```
INSERT INTO adhoc.primary_key_test
VALUES (4), (4);
```

Quick Demo: https://github.com/keeyong/sjsu-data226-FA25/blob/main/week3/PRIMARY_KEY_UNIQ.sql

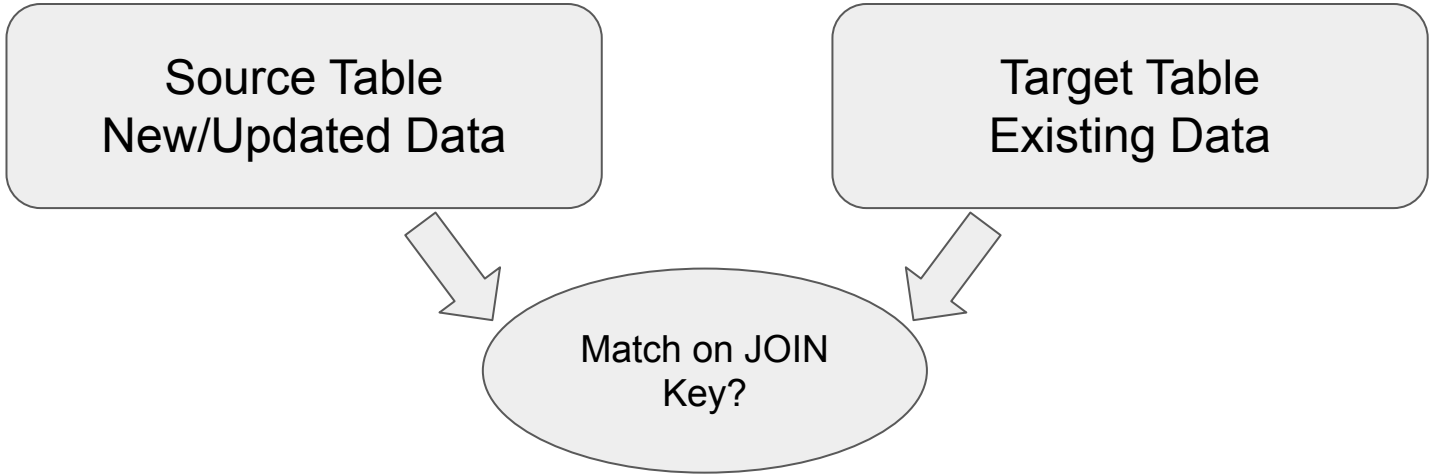
UPSERT Operation in Data Warehouse

- How UPSERT works
  - Operates based on a unique identifier (typically Primary Key or business key):
  - If the record exists: UPDATE with new information
  - If the record doesn't exist: INSERT as a new record
  - For deletions: Use soft deletes by flagging records (e.g., is_deleted boolean field)
- Every Data Warehouse solutions support UPSERT
  - In Snowflake it is called **MERGE** operation
- Critical for Implementing Large Data update in Data Pipelines
  - More on the following slide

Full Refresh vs. Incremental Update

- Full Refresh:
  - Reloads entire dataset from source system
  - Simple but resource-intensive and sometimes not feasible
  - Suitable for small datasets or when change tracking is unavailable
- Incremental Update:
  - Processes only changed/new records since last update
  - Much more efficient for large datasets but more complex
  - Relies on change detection mechanisms (timestamps, change data capture, etc.)
  - Example: **Hourly** updates processing only records modified in the past hour
  - Uses **UPSERT** to handle mixed INSERT/UPDATE operations

MERGE SQL in Snowflake to Support Incremental Update

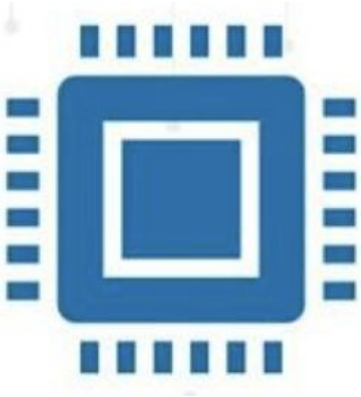


When matched, UPDATE existing records
When not matched, INSERT new records

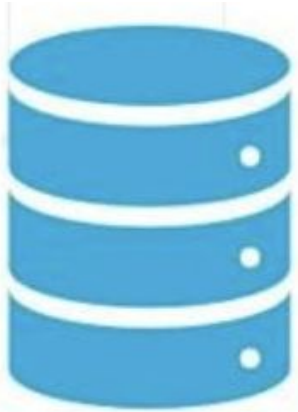
Quick Demo: https://github.com/keeyong/sjsu-data226-FA25/blob/main/week3/MERGE.SQL

Data Warehouse: Compute and Storage

- Storage is about your tables and is cheaper: disk
- Compute is about processing SQL queries and is more expensive: CPU, memory
- Ideally scaling each should be independent



Compute Layer



Storage Layer

Data Warehouse: Compute and Storage

- In some Data Warehouses, this is tightly coupled
  - AWS Redshift is a good example: You purchase compute + storage together
  - You have to scale when you are out of space (disk) as the data becomes too big
  - You also have to scale when you need more computing power
- In some other Data Warehouses, this is decoupled as storage and compute
  - Storage and Compute are independent. You purchase them separately
  - Most variable cost options belong here
    - AWS Redshift Serverless, BigQuery, Snowflake
  - This is more ideal option with a few benefits
    - For example, the data (storage) can be shared in some scenario

Redshift Fixed Cost Options (Virginia Region)

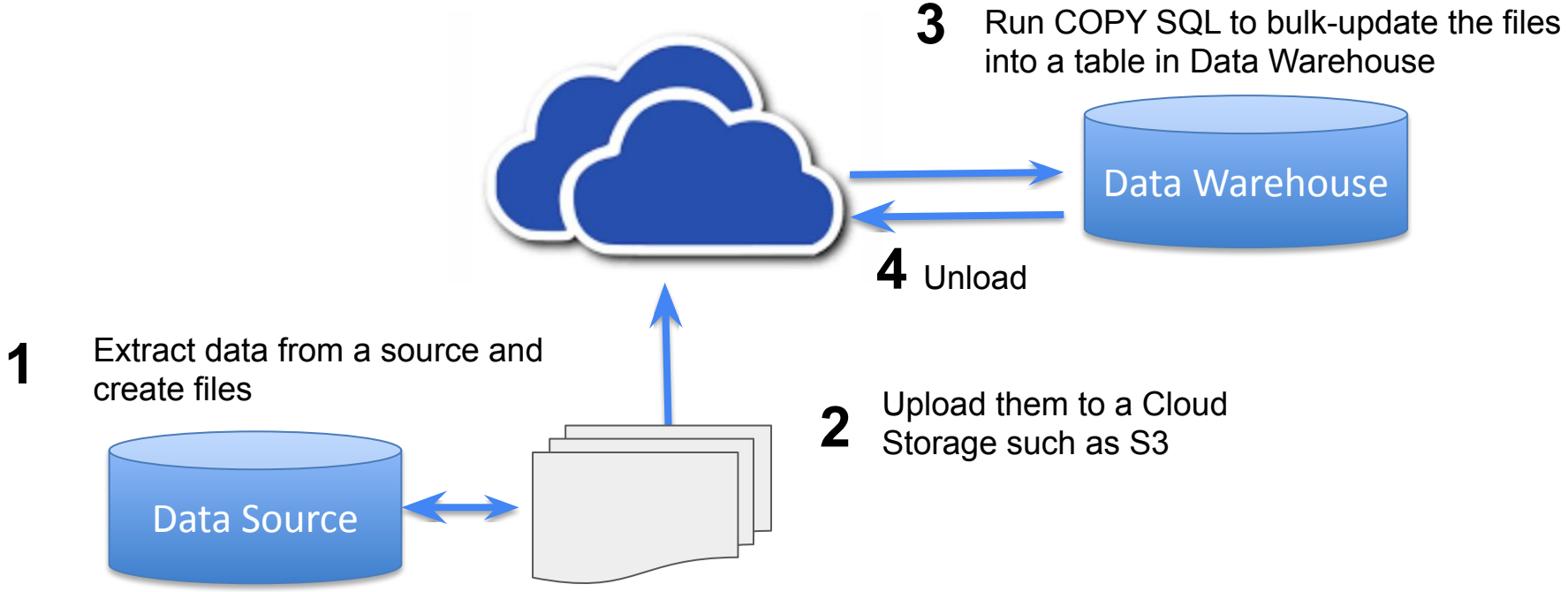
	vCPU	Memory	Storage	Price	Monthly
Dense Storage					
ds2.xlarge	**4**	**31**	**2TB HDD**	**$0.85/hour**	**$612**
ds2.8xlarge	36	244	16TB HDD	$6.80/hour	$4,896
Dense Compute					
dc2.large	**2**	**15**	**0.16TB SSD**	**$0.25/hour**	**$180**
dc2.8xlarge	32	244	2.56TB SSD	$4.80/hour	$3,456
Managed Storage					
ra3.4xlarge	**12**	**96**	**64TB SSD**	**$3.26/hour**	**$2,347**
ra3.16xlarge	48	384	64TB SSD	$13.04/hour	$9,389

Evolution of Data Infra

1. Data Warehouse
2. Data Lake + Data Warehouse
    a. From this point, usually separation of compute and storage is normal
    b. Very specific purpose data warehouses are sometimes introduced (called “Data Mart”)
3. Data LakeHouse

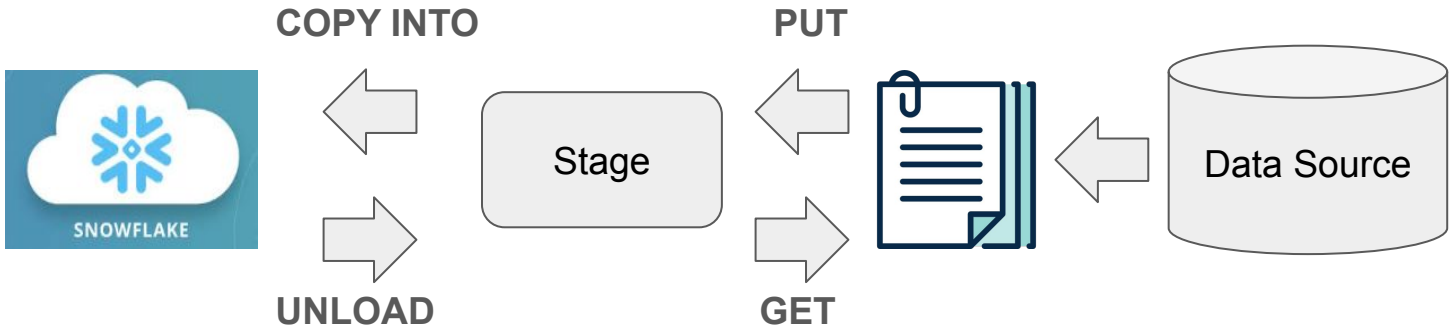
Bulk-update is preferred in Data Warehouse or Data Lake

- What if you have to upload a few million records?
  - This is why DW/DL don't guarantee primary key uniqueness



Bulk Update in Snowflake (COPY INTO)

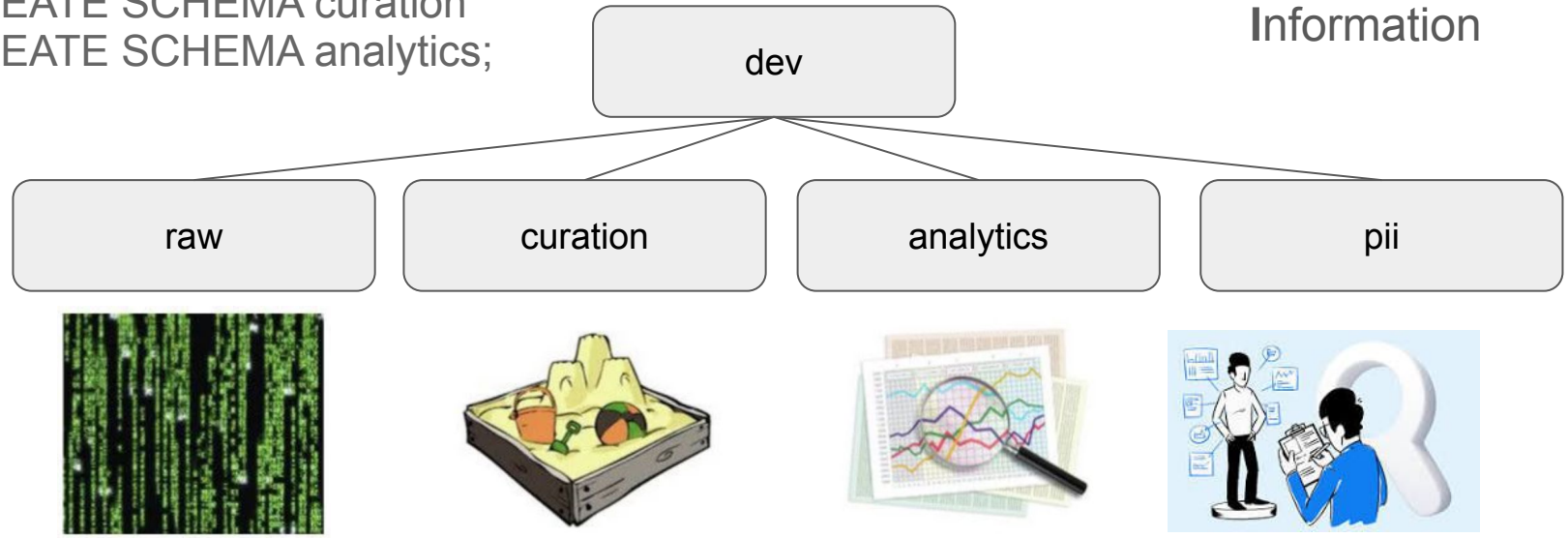
- In Snowflake, the corresponding command is COPY INTO
- This process is done at the file level rather than on a record-by-record basis, using an intermediary called a “stage”
- Conversely, you can also download data from Snowflake to a stage using the UNLOAD command.



Database ➔ Schema: a way to organize your tables

```
CREATE DATABASE dev;
CREATE SCHEMA raw;
CREATE SCHEMA curation
CREATE SCHEMA analytics;
```

Personally
Identifiable
Information



ETL Tables

Cleanup

ELT Tables

PII Tables

Break

WiDS

1. tinyurl.com/sjsuwidsrsvp
2. louisa.stumpf@sjsu.edu

Snowflake Introduction

Snowflake: Data Platform as a Self-managed Service

- Storage layer and Compute layer are separated
- Compute layer is called “Virtual WareHouse”
  - Two types of Virtual WareHouse exist:
    - Regular
    - Snowpark: Programming interface such as Python, Java or Scala
  - A Virtual WH has a size
    - From X-Small to 6X-Large
  - You can have multiple Virtual WHs (more on this later)
  - The same storage layer can be shared among multiple Virtual WH
- Storage layer supports “Time Travel” and “Zero Copy Cloning”
  - SELECT * FROM my_table AT(TIMESTAMP => '2024-08-24 12:00:00');

Snowflake: Data Platform as a Self-managed Service

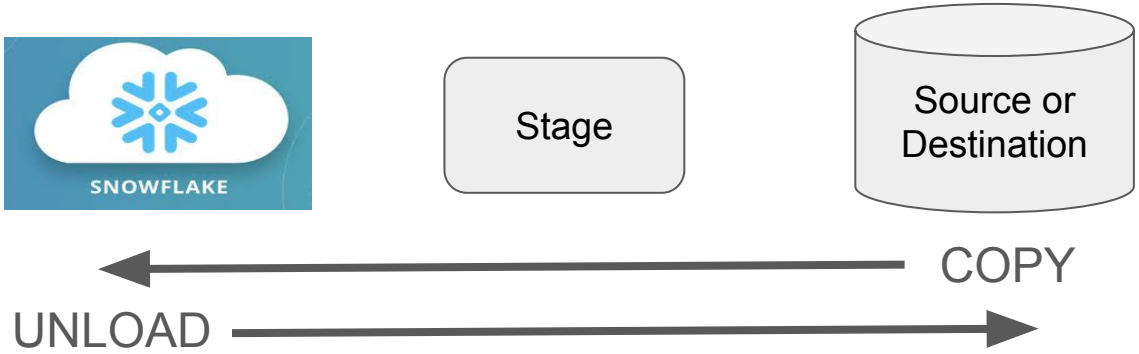
Query Processing Layer: Compute	Virtual Warehouses
Database Layer: Storage	Tables, Schema, …
Cloud Services	Additional Data Services such as Security, Meta data

Features of Snowflake (1)

- Scalable **SQL** engine with **separate storage** and **compute** layers
  - Worksheet (Web UI) or Notebook or SnowSQL (a commandline shell)
- Dataframe based programming is supported called “**Snowpark**”
  - Python, Scala, Java, …
- **Streaming** processing is supported
- **Time Travel** is supported: not available in Standard option
- **Zero Copy Cloning** of storage is possible for development and test
- Cross-Region Replication is supported
- Runs on top of AWS, GCP and Azure

Features of Snowflake (2)

- **Stage:** a middle ground between Snowflake and Source/Destination
  - Internal stage (table/user/named) and external stage
- Cloud storage is supported as **external storages**
  - S3, Google Cloud Storage, Azure Blob Storage (kind of Data Lake)



Features of Snowflake (3)

- Snowflake Account Structure
  - Organization -> 1+ Account -> 1+ Databases
  - An Organization can have multiple Accounts (“department”)
  - An Account can have multiple databases and multiple users
  - A Database can scale up to PB and is composed of schemas and tables
- Data Marketplace
  - You can advertise your data and can sell it
- Data Sharing (“Share, Don’t Move”)
  - “Data Sharing” enables you to share data not copy/clone it at the storage level

Snowflake Data Types

- Numeric: TINYINT, SMALLINT, INTEGER, BIGINT, NUMBER, NUMERIC, DECIMAL, FLOAT, DOUBLE, REAL.
- Boolean: BOOLEAN.
- String: CHAR, VARCHAR, TEXT, BINARY, VARBINARY.
- Date and Time: DATE, TIME, TIMESTAMP, TIMESTAMP_LTZ, TIMESTAMP_TZ.
- Semi-structured data: VARIANT (JSON, OBJECT)
- Binary: BINARY, VARBINARY
- Geospatial: GEOGRAPHY, GEOMETRY
- Array: ARRAY
- Object: OBJECT

Snowflake Pricing

- Snowflake has 4 options
  - Standard, Enterprise, Business Critical and Virtual Private Snowflake
- Pricing itself has 3 components
  - Compute Costs: Determined by credits (more on this later)
  - Storage Costs: Calculated per terabyte (TB). This includes stage
  - Network Costs: Calculated per TB for data transfers between regions or across different cloud providers

Snowflake Compute Pricing

- Credit: a unit of measurement used to determine the cost of computing resources (running SQL queries and other computations)
- Cost of 1 credit depends on the pricing option
  - Standard: $2
  - Enterprise: $3
  - Business-critical: $4
- Virtual Warehouse size will determine credit consumption
  - X-Small: 1 credit / hour
  - …
  - 6X-Large: 512 credits / hour

Snowflake Pricing per Warehouse Size

Warehouse Size	Credits / Hour	Credits / Second	Notes
X-Small	1	0.0003	Default size for warehouses created using CREATE WAREHOUSE.
Small	2	0.0006	
Medium	4	0.0011	
Large	8	0.0022	
X-Large	16	0.0044	Default for warehouses created in the web interface.
2X-Large	32	0.0089	
3X-Large	64	0.0178	
4X-Large	128	0.0356	
5X-Large	256	0.0711	Preview feature.
6X-Large	512	0.1422	Preview feature.

Lab #1 Overview

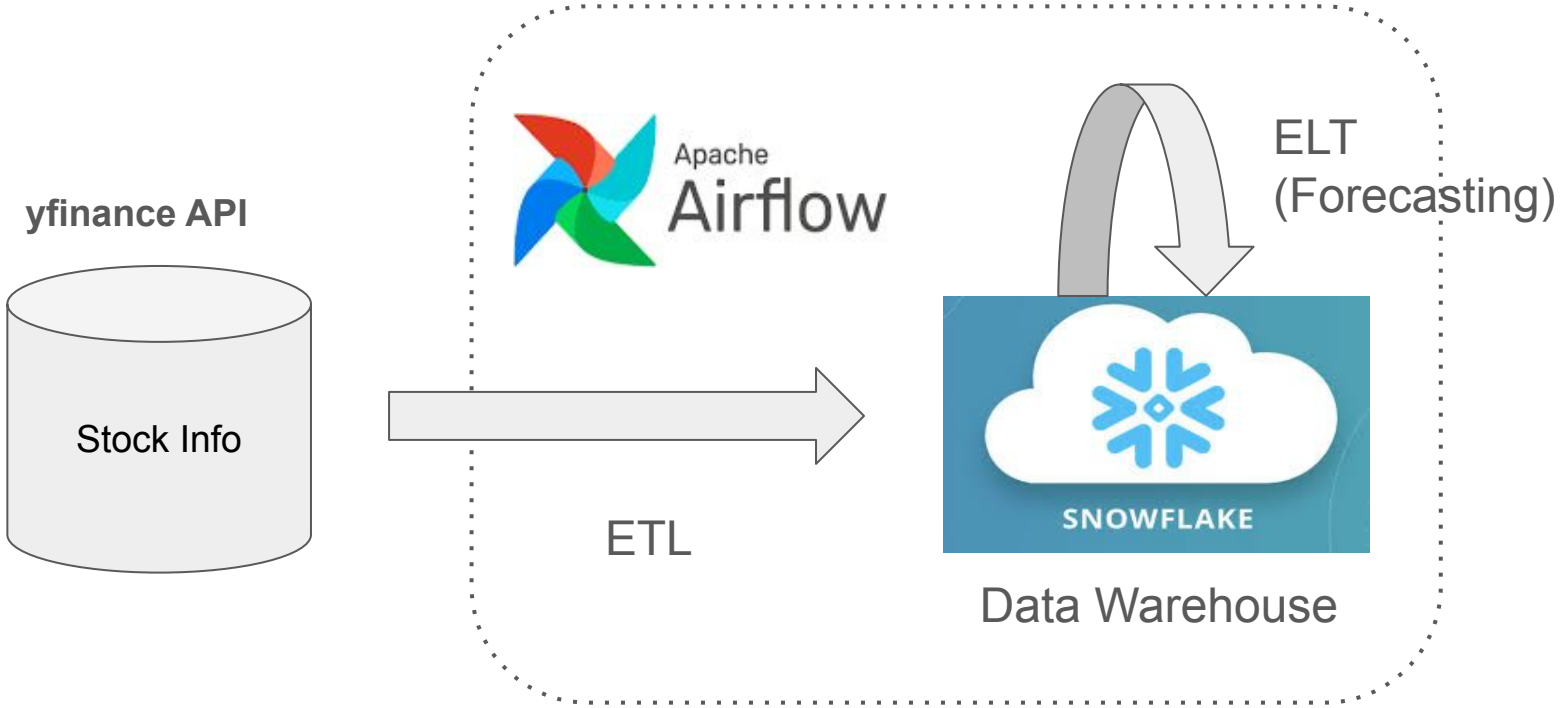
Section 24: Building a Finance Data Analytics using Snowflake & Airflow

Section 23: Building a Finance Data Analytics using Snowflake & Airflow

This is for a team of two students

Due by 10/07

- Complete the team selection by EOD today (09/09)
- Due date is 10/07 3PM (sec 24) or 6PM (sec 23)



Quiz #1 Format

Details

- 15 multi-choice questions in 15 minutes
  - The same format as the last Assessment Survey
- Lockdown browser with webcam: make sure your laptop is ready
- Week 1 to Week 4 will be covered
- “In-person” quiz

What is the purpose of a data lake in a modern data architecture?

To store raw data in its native format	31 respondents	**72 %**
To store only structured data	4 respondents	9 %
To replace traditional databases	6 respondents	14 %
To perform real-time analytics	2 respondents	5 %

Demo & Homework

Demo

- Primary key constraint is just a hint in Data Warehouse
- MERGE SQL enables conditional insert, update, and delete operations

MERGE INTO syntax

```sql
MERGE INTO target_table AS target
USING staging_table AS stage
ON target.id = stage.id
WHEN MATCHED THEN
    UPDATE SET
        target.col1 = stage.col1,
        target.col2 = stage.col2
WHEN NOT MATCHED THEN
    INSERT (col1, col2, col3)
    VALUES (stage.col1, stage.col2, stage.col3);
```

The table you want to update

The new data (typically smaller table)

Merge condition

What to do if a row already exists (update)

What to do if a row doesn’t exist (insert)

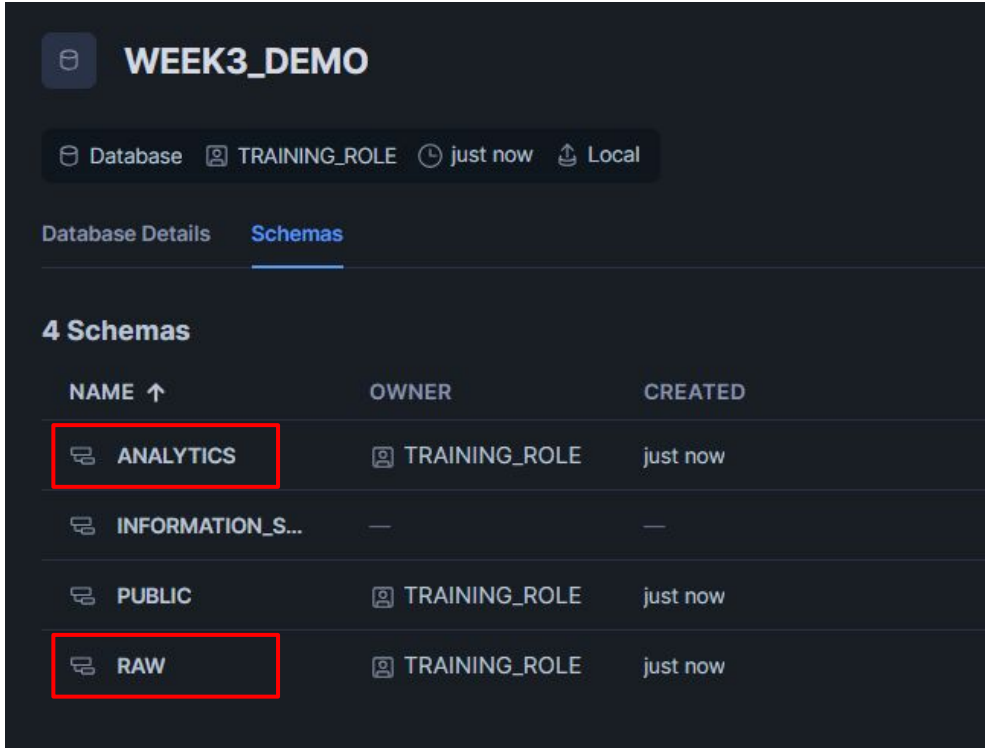
Homework #2 Details (12 points)

Build a basic data pipeline using COPY INTO and MERGE operations to simulate incremental data loading.

- Use prod_old_data.csv, prod_new_data.csv files are under *Files/Homeworks/*
- Please follow the demo and complete the below assignment

1. (+1) Make sure you have schemas RAW and ANALYTICS under your database
2. (+3) In RAW schema, create prod_new_data table
    a. Proper table schema design (+1)
    b. COPY INTO needs to be used. This time, SQL is preferred (not UI) (+2)
        i. prod_new_data.csv should be the input
3. (+2) Repeat the same for prod_old_data table with prod_old_data.csv
4. (+3) Merge prod_new_data table into prod_old_data using MERGE SQL
5. (+3) Provide a written explanation of the end-to-end process from an incremental update perspective

Screenshot Examples for HW2 Step 1



Screenshot Examples for HW2 Steps 2 & 3

Provide SQL code in submission (create table, file format and copy into) and:

```
25  COPY INTO prod_new_data
26  FROM @RAW.PROD_NEW_STAGE
27  FILE_FORMAT = (FORMAT_NAME = 'prod_data_file_format');
28
```

Results | Chart

	file	status	rows_parsed	rows_loaded	error_limit	errors_seen
1	prod_new_stage/prod_	LOADED	4620	4620	1	0

Screenshot Examples for HW2 Step 4

Provide the MERGE INTO code and:

Results Chart

	number of rows inserted	number of rows updated
1	4620	0

Resume

- Audience of your resume is not YOU but hiring managers and recruiters
- Readability matters
  - Work Experience and Projects should be more like telling your story
    - Not a list of what you have done!!
  - Categorize your Skills
  - Customize and optimize for the job you are applying for!
    - Look for frequent keywords in the job description
- Be friends with ChatGPT

Q & A

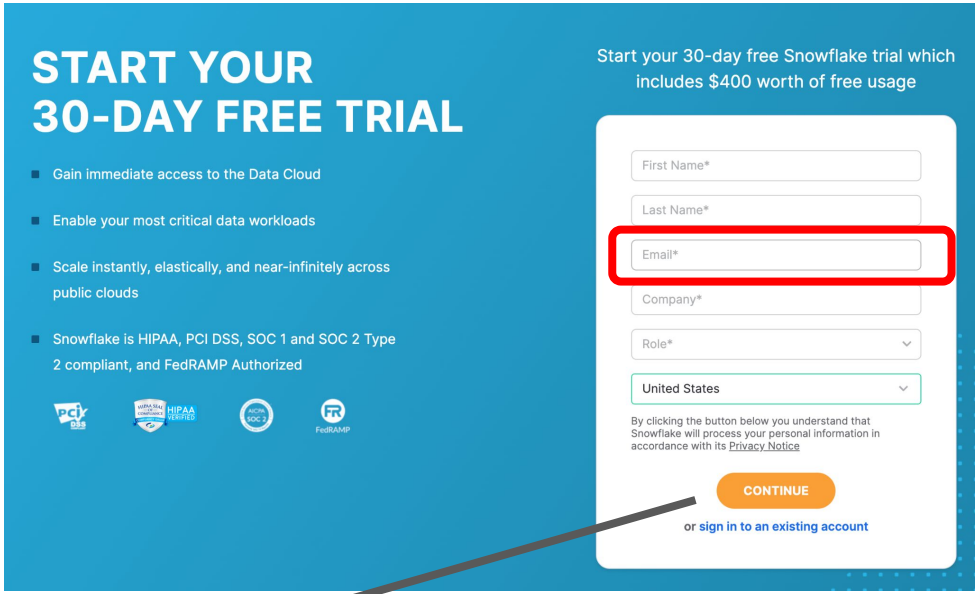
Next Week: Advanced SQL

We will explore advanced SQLs essential for real-world applications

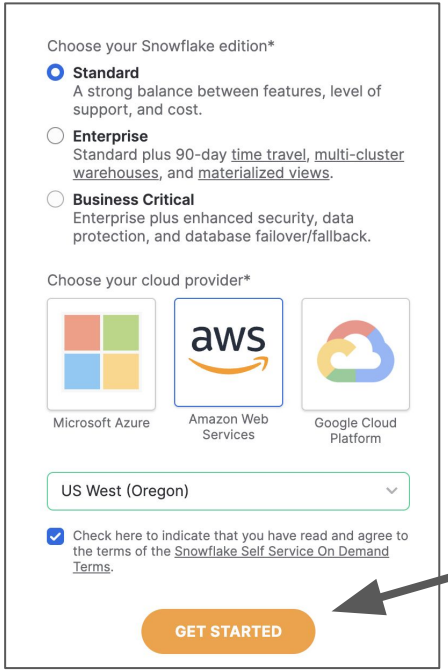
Archived

Snowflake Free Trial (1): https://signup.snowflake.com/

- 30 days or max $400 dollars
- Standard Option



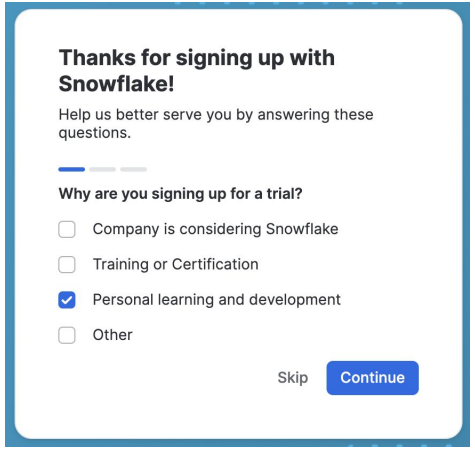
1



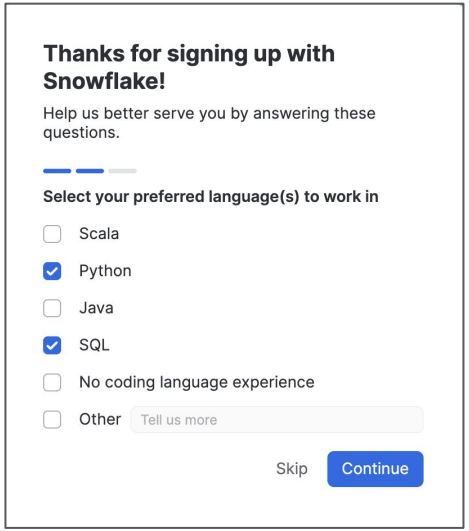
2

Snowflake Free Trial (2)

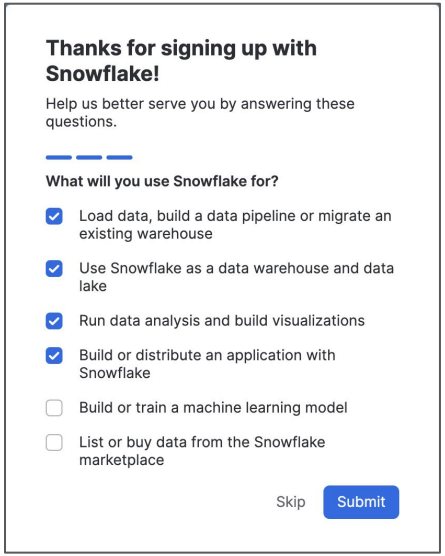
3



4



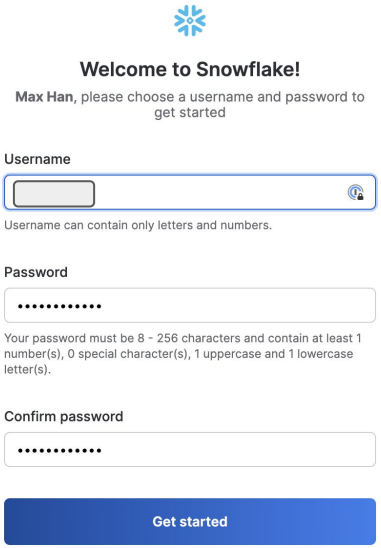
5



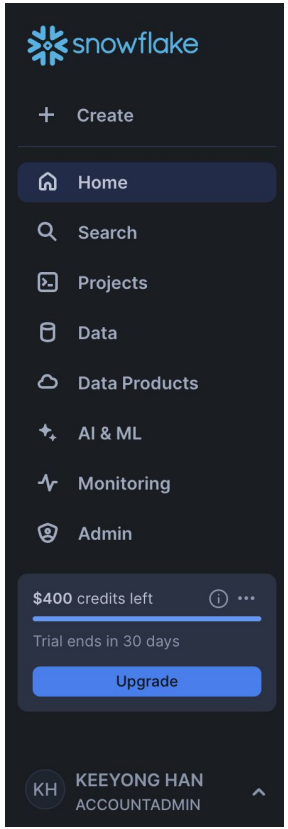
Check your email
(“Activate your Snowflake account”)

Snowflake Free Trial (3)

6



7



8. Remember your login URL for the next login (“Save this for later” section of the activation email)

How to End Your Free Trial

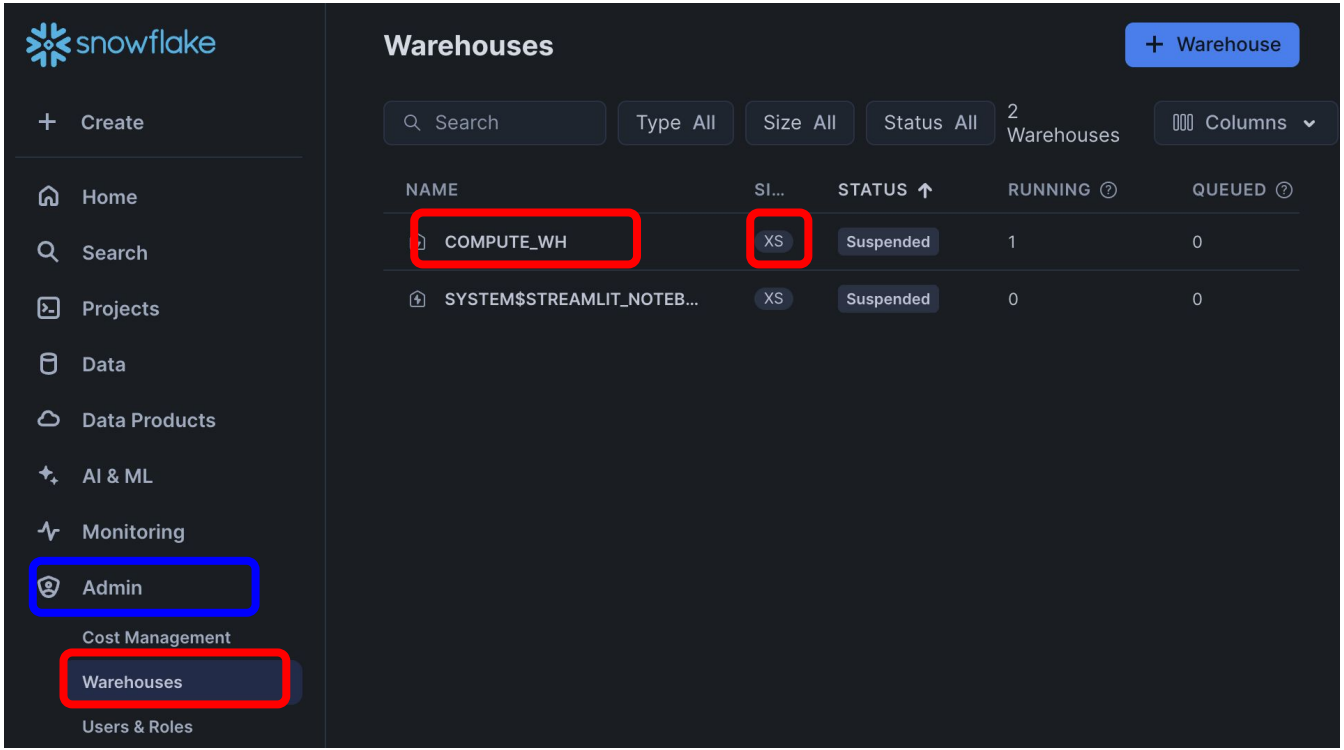
- Once the trial ends, the account will be in “Suspended Mode”
  - To activate, you have to enter your credit card info
- If you want to end it earlier, you can email to Snowflake support
  - https://docs.snowflake.com/en/user-guide/admin-trial-account#canceling-a-trial-account

Check pre-created databases, schemas and tables

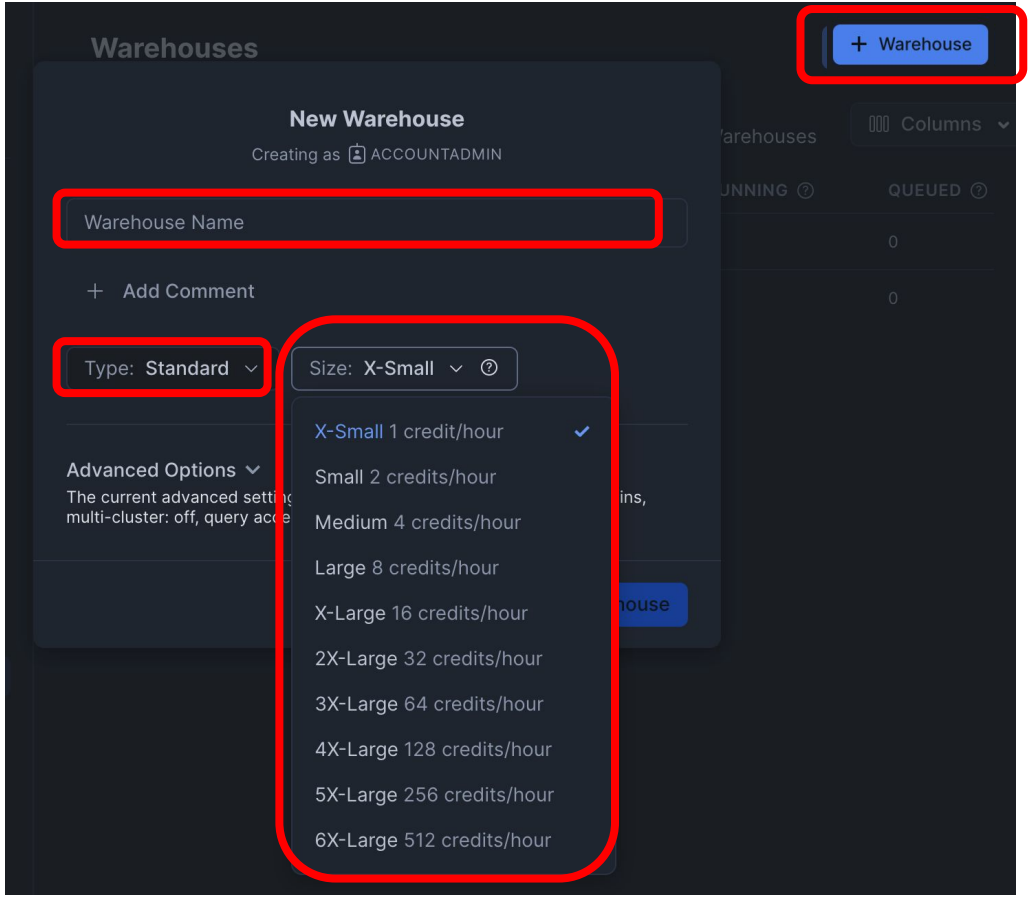
- Data -> Databases: there are two databases

Check the pre-created Warehouses

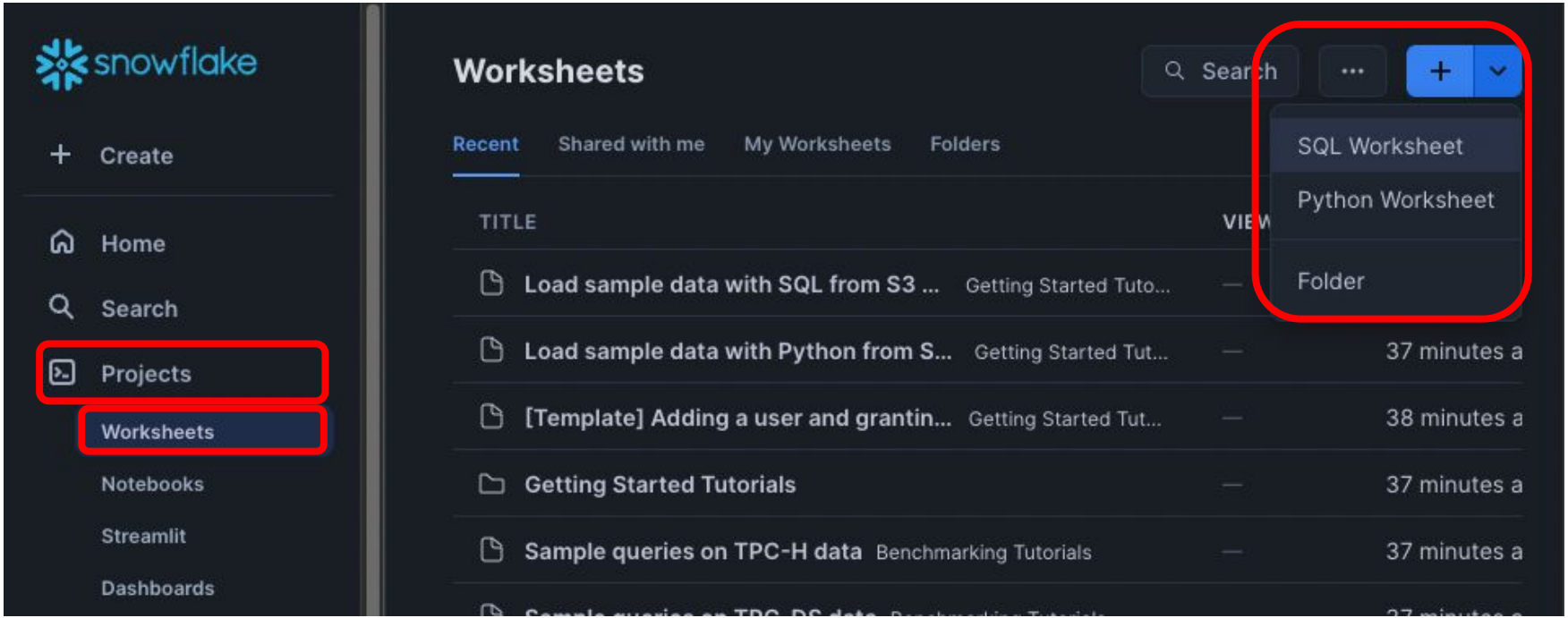
- Admin -> Warehouses: COMPUTE_WH (XS: X-Small)



How to Create a new Warehouse (“+ Warehouse”)

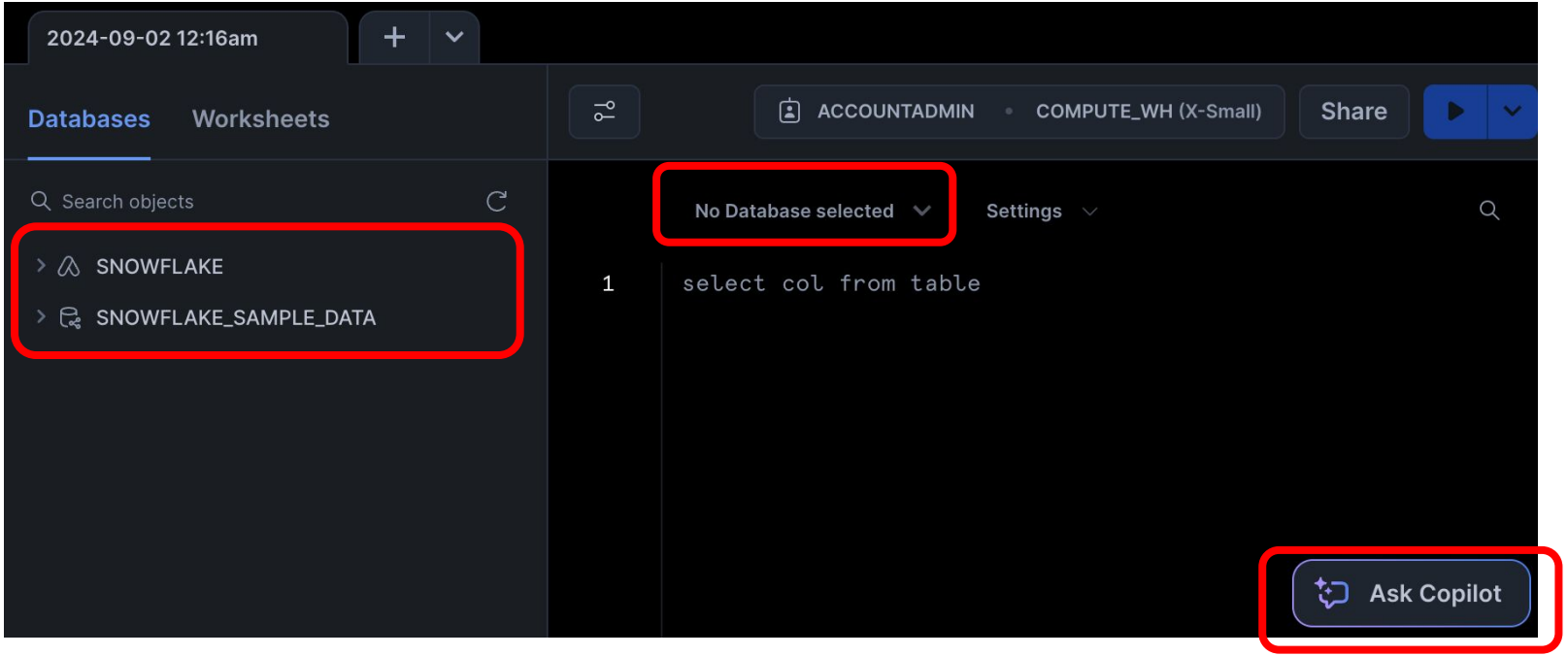


Run SQLs on Worksheet (1)

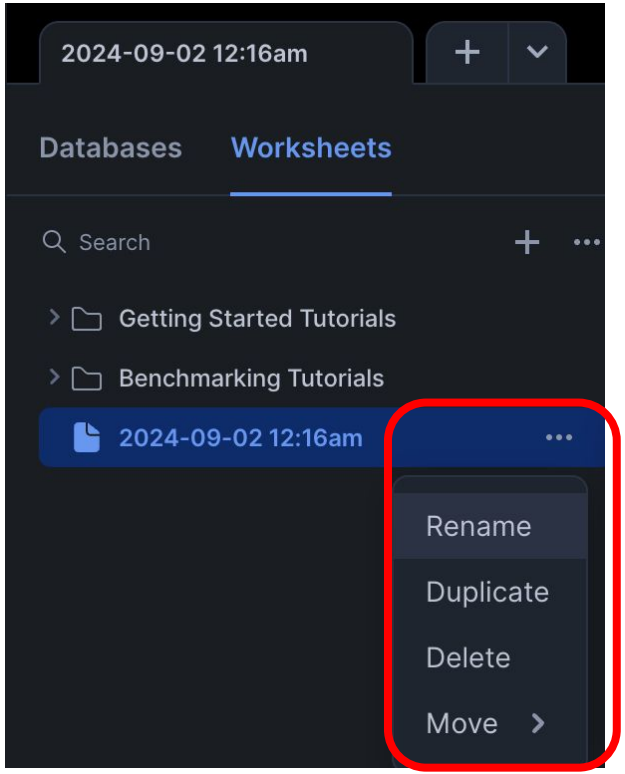


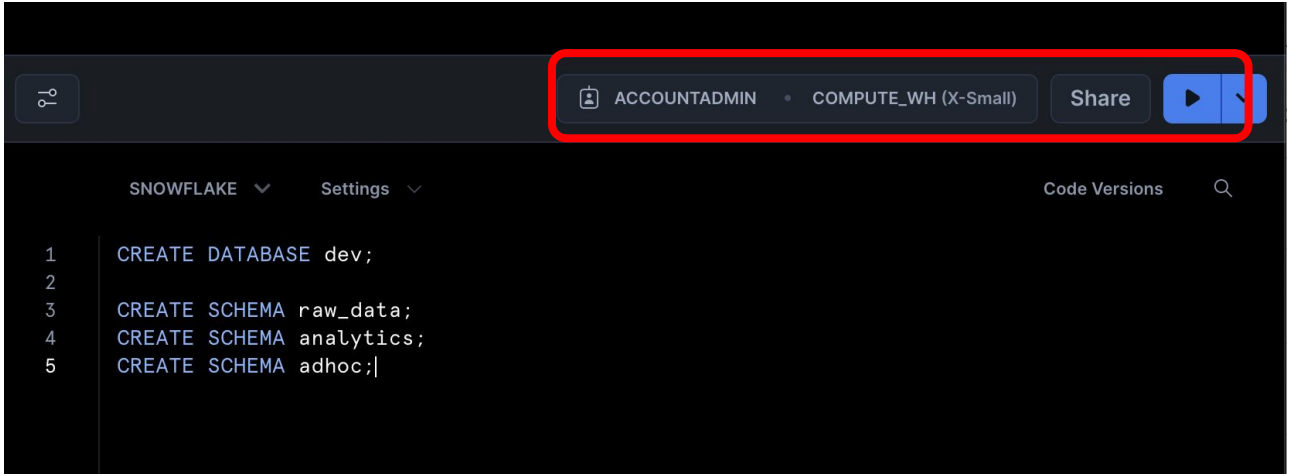
Run SQLs on Worksheet (2)

- Make sure to select the right database to work on or to create a new one



Run SQLs on Worksheet (3)

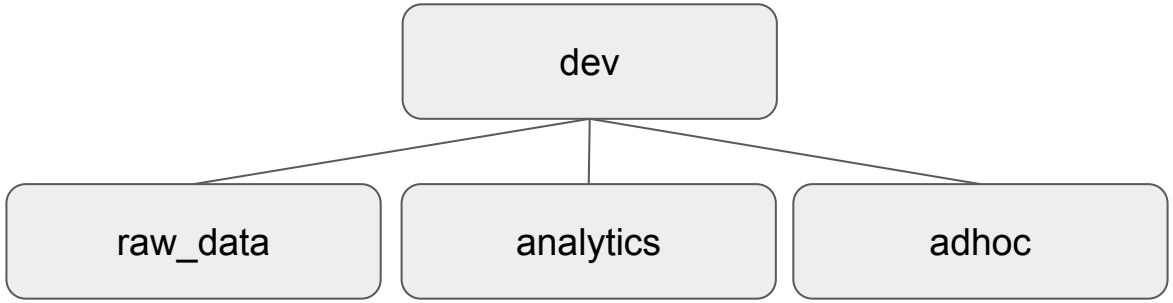




Run SQLs on Worksheet (4)

```
CREATE DATABASE dev;

CREATE SCHEMA raw_data;
CREATE SCHEMA analytics;
CREATE SCHEMA adhoc;
```



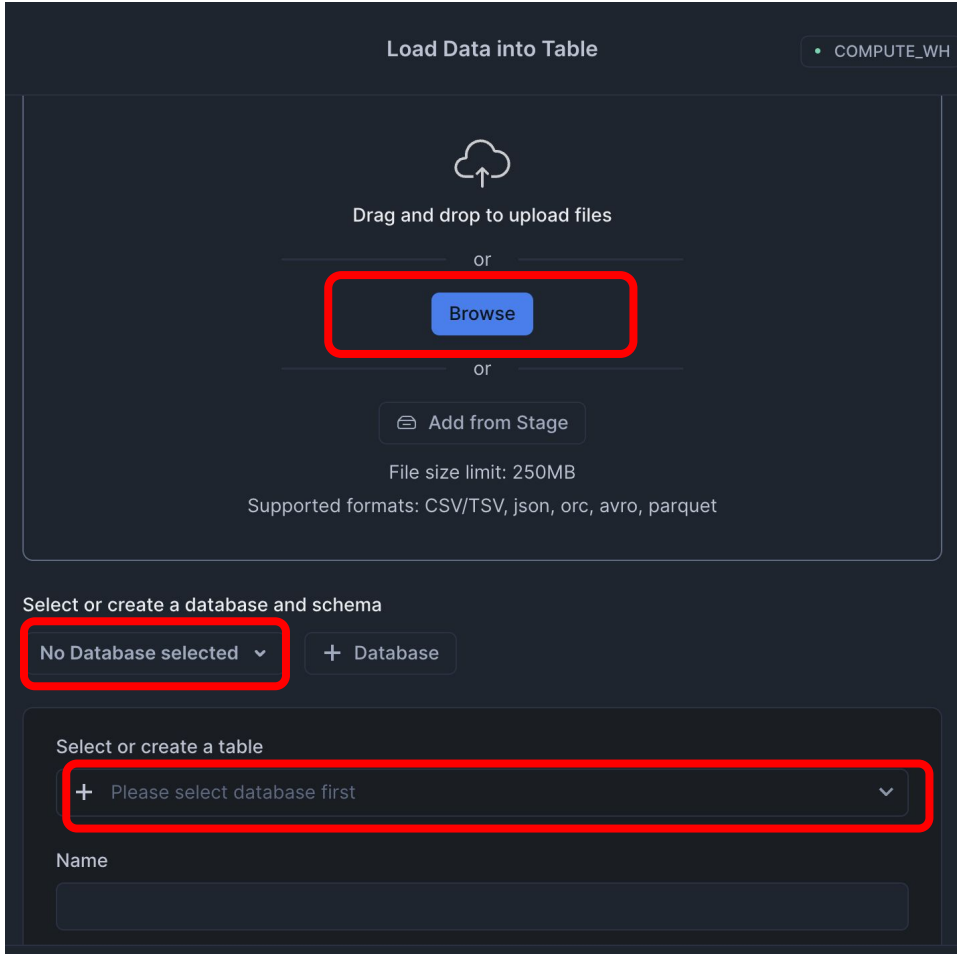
Run SQLs on Worksheet (5)

- Create DEV.raw_data.nps table as following

```
CREATE OR REPLACE TABLE DEV.raw_data.nps (
    id int PRIMARY KEY,
    created timestamp,
    score smallint
);
```

Run SQLs on Worksheet (6)

- Populate the table with nps.csv file from demo “02-Snowflake” folder
  - Download the file from the Canvas first to your laptop
- Next upload it directly to DEV.raw_data.nps table
  - Data -> Add Data -> “Load Data into Table”
  - With SnowSQL, you can use stages



Run SQLs on Worksheet (7)

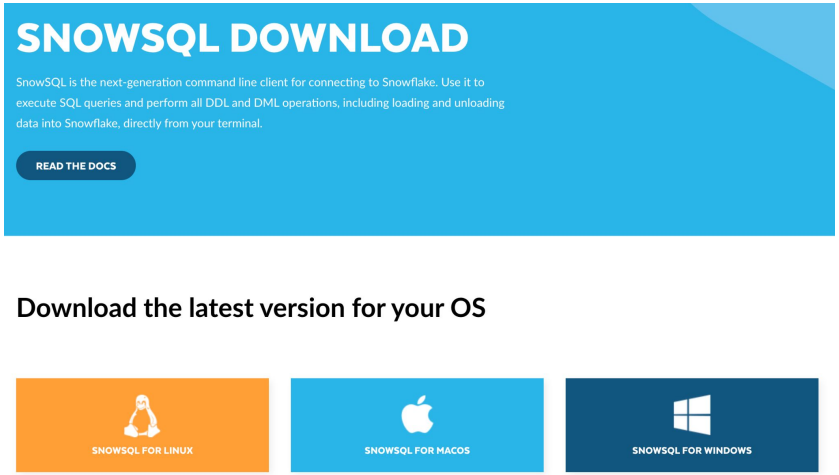
- Create Monthly NPS **ELT** table using CTAS

```
CREATE TABLE DEV.ANALYTICS.monthly_nps AS
SELECT
   LEFT(created, 7) month,
   AVG(score) nps
FROM DEV.raw_data.nps
GROUP BY 1;
```

Run the SQLs on SnowSQL (Command Line Tool)

- Install SnowSQL (https://developers.snowflake.com/snowsql/)
- Run snowsql from terminal
  - snowsql -a **account_name** -u user_name

You can extract the account name from the snowflake login url



Redshift Introduction

What is Redshift?

- OLAP: Data warehouse service supported by AWS
  - Can process up to 2 PB of data
  - Cannot be used as a production database due to slower response times
- Column-based storage
- Does not guarantee primary key uniqueness
- Supports bulk updates
  - Upload files containing records to S3, then use the **COPY** command to bulk-load to Redshift

What is Redshift?

- Fixed capacity/cost SQL engine by default
  - Recently introduced variable cost option (**Redshift Serverless**)
- Data sharing feature (Datashare):
  - Ability to share specific data with other AWS accounts
  - Snowflake started this first and AWS copied
- Remember once again that SQL is the main language
  - Data modeling (table design) is very important
    - Dimensional Modeling vs. Denormalized Modeling
    - More on this later during dbt/ELT class

Redshift is still a SQL based Relational Database

- Redshift SQL is compatible with PostgreSQL 8.x
  - However, it doesn't support all features of PostgreSQL 8.x
  - For example, the JSON type doesn't exist in Redshift
- Can be accessed using tools or libraries that support PostgreSQL 8.x
  - For Python coding, use **psycopg2** module

Redshift vs. Redshift Serverless

- Redshift: **Fixed** Cost Option
  - You purchase a predefined capacity based on the instance types and pay a fixed fee
  - Here capacity means both “storage” (disk) and “computation” (CPU/memory)
    - Storage and computation are tightly coupled
  - “Auto Scaling” option: only scales compute not the storage
- Redshift Serverless: **Variable** Cost Option
  - Pay-per-Use Pricing
    - Pay only for the resources used while queries are running + storage cost of your data (tables)
  - Separation of data and compute resource
    - Compute resources are allocated only when needed for query processing

Redshift Fixed Cost Options (Virginia Region)

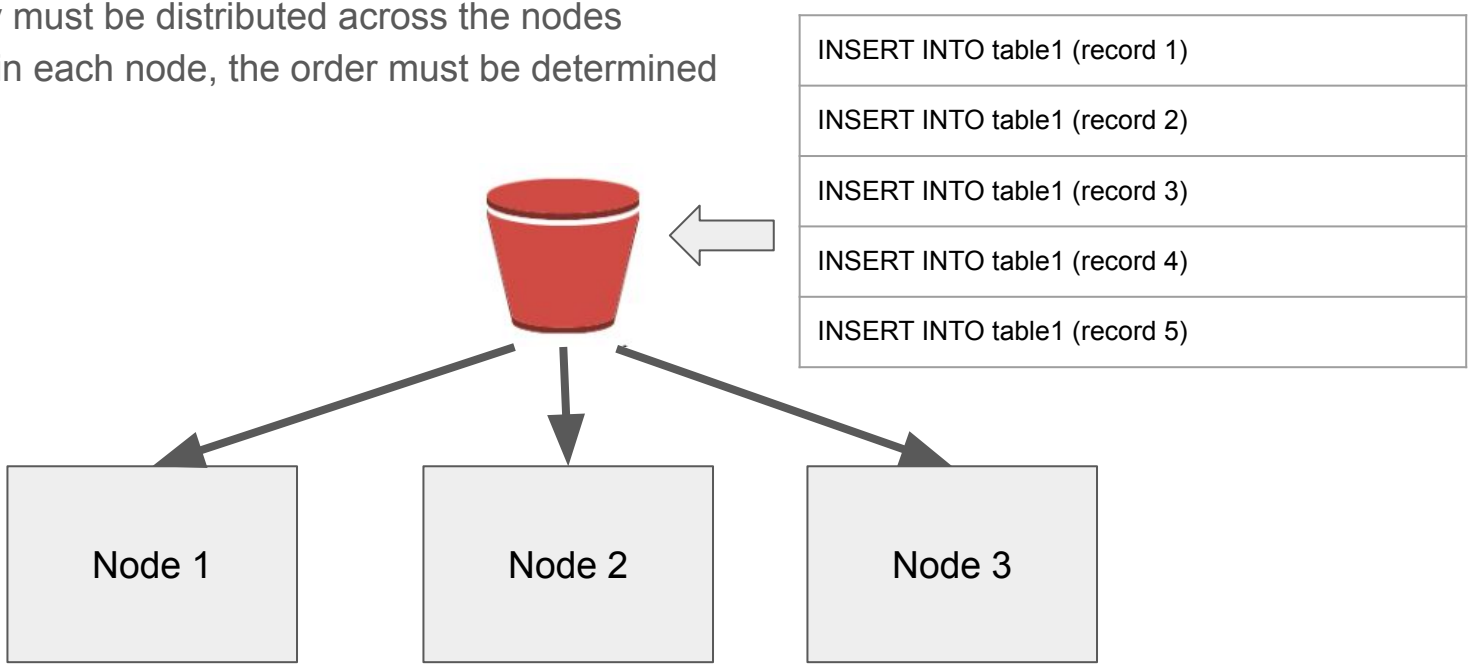
	vCPU	Memory	Storage	Price	Monthly
Dense Storage					
ds2.xlarge	**4**	**31**	**2TB HDD**	**$0.85/hour**	**$612**
ds2.8xlarge	36	244	16TB HDD	$6.80/hour	$4,896
Dense Compute					
dc2.large	**2**	**15**	**0.16TB SSD**	**$0.25/hour**	**$180**
dc2.8xlarge	32	244	2.56TB SSD	$4.80/hour	$3,456
Managed Storage					
ra3.4xlarge	**12**	**96**	**64TB SSD**	**$3.26/hour**	**$2,347**
ra3.16xlarge	48	384	64TB SSD	$13.04/hour	$9,389

Scaling Redshift (Not Serverless): Resizing

- Basically Scale Out
  - Scales by adding new nodes whenever capacity becomes insufficient
- But “Scale Up” can be done together
  - 1 dc2.large instance gives you 0.16TB disk space
  - When out of capacity
    - Add a new dc2.large instance: now you have total 0.32TB (Scale Out)
    - Or you upgrade from dc2.large to dc2.8xlarge (Scale Up)
    - Or do both (Scale Out and Scale Up)
- Redshift has “Auto Scaling” option
  - It is only about “computing nodes”. Storage needs manual changes

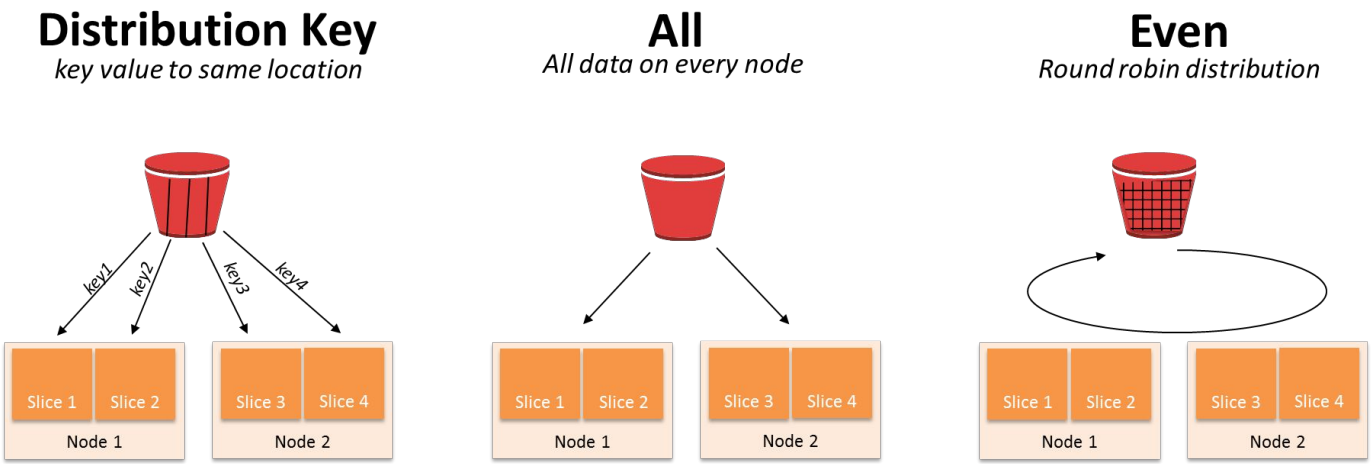
Redshift (Not Serverless): Records Distribution

- When Redshift has 1+ node, how are the records of a table stored?
  - They must be distributed across the nodes
  - Within each node, the order must be determined



Redshift (Not Serverless): diststyle, distkey, and sortkey

- Diststyle: all, even, key (default is “even”)
- Distkey determines which column to use for record distribution
  - when diststyle is “key”
- Sortkey determines how records are ordered in a node
  - Usually a timestamp field is picked



Redshift Example of diststyle, distkey, and sortkey

```
CREATE TABLE my_table (
    column1 INT,
    column2 VARCHAR(50),
    column3 TIMESTAMP,
    column4 DECIMAL(18,2)
) DISTSTYLE KEY DISTKEY(column1) SORTKEY(column3);
```

Records in my_table will be distributed based on column1
Records in the same node will be ordered by column3

Redshift Data Types

- SMALLINT (INT2)
- INTEGER (INT, INT4)
- BIGINT (INT8)
- DECIMAL (NUMERIC)
- REAL (FLOAT4)
- DOUBLE PRECISION (FLOAT8)
- BOOLEAN (BOOL)
- CHAR (CHARACTER)
- **VARCHAR (CHARACTER VARYING)**
- TEXT (VARCHAR(256))
- DATE
- TIMESTAMP

- GEOMETRY
- GEOGRAPHY
- HLLSKETCH
- SUPER

Redshift Recommendation

- With fixed cost option
  - You have to scale when out of compute or storage
  - You have to optimize the distribution
- 1-node Redshift is fine (and Scale Up) as long as the capacity is fine
- Multi-node Redshift isn’t good
  - You have to scale manually
  - You have to optimize the record distribution (distkey, diststyle, …): This is a big **PAIN**
- Redshift Serverless is a better option
  - Variable Cost Option is more scalable
  - Optimizing the distribution isn’t bad

Overall Google Cloud BigQuery or Snowflake is a better option

Redshift Practice

ELT Table Creation using CTAS

- Use analytics schema
- Use SQL CTAS (Create Table … As SELECT)

```
CREATE TABLE analytics.monthly_nps AS
SELECT
   LEFT(created, 7) "month",
   AVG(score) nps
FROM adhoc.nps
GROUP BY 1;
```

NPS: Net Promoter Score

Survey on a 0 to 10 scale
Promoter: 9,10
Passive: 7, 8
Detractor: 0-6

NPS=Promoter % - Detractor %

Practice Google Colab

https://colab.research.google.com/drive/1xNKDWAURCjjpfbcG1fjOIlH0de9XKY42

Assessment Survey Results: Avg 85%

- Make sure your lockdown browser is working before week 4 (Quiz)

Attempts: 43 out of 43

+0.57

What is the primary purpose

Social media management
Online customer interactions
Real-time transaction processing
Data

What is the purpose of a data lake in a modern data architecture?

To store raw data in its native format	31 res

What is a key characteristic of a star schema in Data Warehousing?

	espondents	19 %
	respondents	30 %
	respondents	**49 %** ✓
	espondent	2 %

In Python, which of the following is used to handle exceptions/errors?

while loop		0 %
if-else	6 respondents	14 %
try-except	37 respondents	**86 %** ✓
for loop		0 %